

COMMON COURSE ASSIGNMENT
CSA6502 — GENERATIVE AI AND LARGE-SCALE MODELS

Design and Implementation of “IndustroSense AI”

**A Multimodal Responsible Generative AI Application for Industrial Equipment
Diagnostics, Integrating RAG, Vector Databases, AI Agents, and Web Deployment**

Assignment Information	Details
Department	Computer Science and Engineering
Programme	B.E. / B.Tech — CSE
Course	CSA6502 — Generative AI and Large-Scale Models
Academic Year / Batch	2026–2027
Faculty Name	Dr. Nithisha J
Course Outcomes	CO3, CO4 & CO5
Bloom’s Levels	L3 — Apply; L4 — Analyze; L5 — Evaluate; L6 — Create
Maximum Marks	100
SDG Mapping	SDG 4 — Quality Education; SDG 9 — Industry, Innovation and Infrastructure; SDG 12 — Responsible Consumption and Production
RAG	MULTIMODAL
AGENT	RESPONSIBLE AI

Prepared as a professional solution based on the supplied CSA6502 assignment brief.

TABLE OF CONTENTS

Section No.	Section / Topic	Page No.
1	Problem Understanding and Formulation	3–4
2	System Objectives	5
3	Proposed System Architecture	6–7
4	Part I — RAG, Vector Database and AI Agents	8–12
4.1	Question 9 — RAG Pipeline Design	8–9
4.2	Question 10 — Embedding, Indexing and Retrieval Demonstration	9–11
4.3	Question 11 — AI Agent Design	11–12
4.4	Question 12 — Vector Database Indexing Strategies	12
5	Part II — Multimodal Generative AI Integration	13–15
5.1	Question 13 — Multimodal Architecture	13
5.2	Question 14 — End-to-End Multimodal Example	14
5.3	Question 15 — Engineering Challenges	15
6	Part III — Responsible Deployment, Security and Governance	16–19
6.1	Question 16 — Web Deployment Architecture	16
6.2	Question 17 — Responsible-AI Risks and Mitigation	17
6.3	Question 18 — Security Considerations	17–18
6.4	Question 19 — Governance Considerations	18–19
7	Consolidated Solution / Design / Methodology	20
8	Modern Tools and Implementation Plan	21–22
9	Results and Validation	23–24
10	Analysis and Engineering Decisions	25–26
11	Broader Considerations	27
12	Conclusion	28
13	Student Reflection	29
14	References	30
Appendix A	Final Submission Checklist	Not present in the PDF

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 Problem Understanding

Industrial field engineers and plant technicians frequently need immediate assistance when diagnosing equipment faults. However, relevant information is often distributed across equipment manuals, Standard Operating Procedures (SOPs), maintenance records and previous incident reports. Faults may also be reported using different modalities such as written descriptions, photographs and voice notes.

IndustroSense AI is proposed as a unified multimodal Generative AI assistant that retrieves relevant technical knowledge, understands text, images and speech, applies a controlled AI-agent decision process, and generates traceable diagnostic guidance. The system is designed as decision support rather than an autonomous replacement for qualified engineering personnel.

1.2 Major Challenges

Challenge	Proposed Solution
Technical information is scattered	RAG-based centralized knowledge retrieval
Keyword search may miss related concepts	Semantic embeddings and vector similarity
Large document corpus	Vector database indexing
Different user input types	Multimodal processing
Voice reports	Speech-to-text transcription
Equipment photographs	Vision-language analysis
AI hallucination	Retrieval-grounded generation and evidence thresholds
Unsafe recommendations	Confidence checks and human review
Unauthorized access	Authentication and access control
Prompt attacks	Input validation and prompt-injection safeguards
API-key leakage	Environment variables / secrets management
Lack of traceability	Source citations, agent trace and audit logs

1.3 Expected Outcomes

1. Grounded responses based on retrieved technical documents.
2. Reduced hallucination through retrieval-constrained generation.
3. Multimodal fault understanding using text, image and speech.
4. Traceable answers through visible source citations.
5. Explainable agent decisions through a decision trace.
6. Secure web deployment with input validation and access controls.
7. Human oversight for uncertain or safety-critical recommendations.

8. Auditable operation through structured logs.

1.4 Assumptions

Parameter	Assumption	Justification
Document corpus	5–20 sample technical documents	Suitable for semester-scale demonstration
Chunk size	Approximately 500 words	Preserves useful technical context
Chunk overlap	Approximately 75 words	Reduces boundary information loss
Embedding model	all-MiniLM-L6-v2	Lightweight semantic-search model
Embedding dimension	384	Model-defined vector size
Vector database	FAISS	Simple, fast and suitable for a small corpus
Retrieval	Top-3 semantic similarity	Balances evidence coverage and prompt size
Image formats	JPG / JPEG / PNG	Common image formats
Audio formats	WAV / MP3 / M4A	Common classroom/demo formats
Image size limit	5 MB	Controls resource use and abuse
Audio size limit	10 MB	Controls latency and storage
User roles	Technician / Administrator	Separates operational and management access
Review threshold	Confidence < 0.70	Illustrative starting point; calibrate using real validation data
Target latency	A few seconds for representative inputs	Aligned with classroom demonstration requirements

1.5 Constraints

- Python with Streamlit (or an equivalent lightweight web framework).
- Accessible vector database such as FAISS, Chroma or Pinecone.
- Accessible LLM, embedding, vision and speech models/APIs.
- Student-level computing resources and free-tier/trial access where applicable.
- Graceful handling of malformed inputs, missing modalities and API failures.
- Responsible handling of uploaded images, voice recordings and technical information.
- Submission of working code, report, screenshots, evaluation and GitHub repository link.

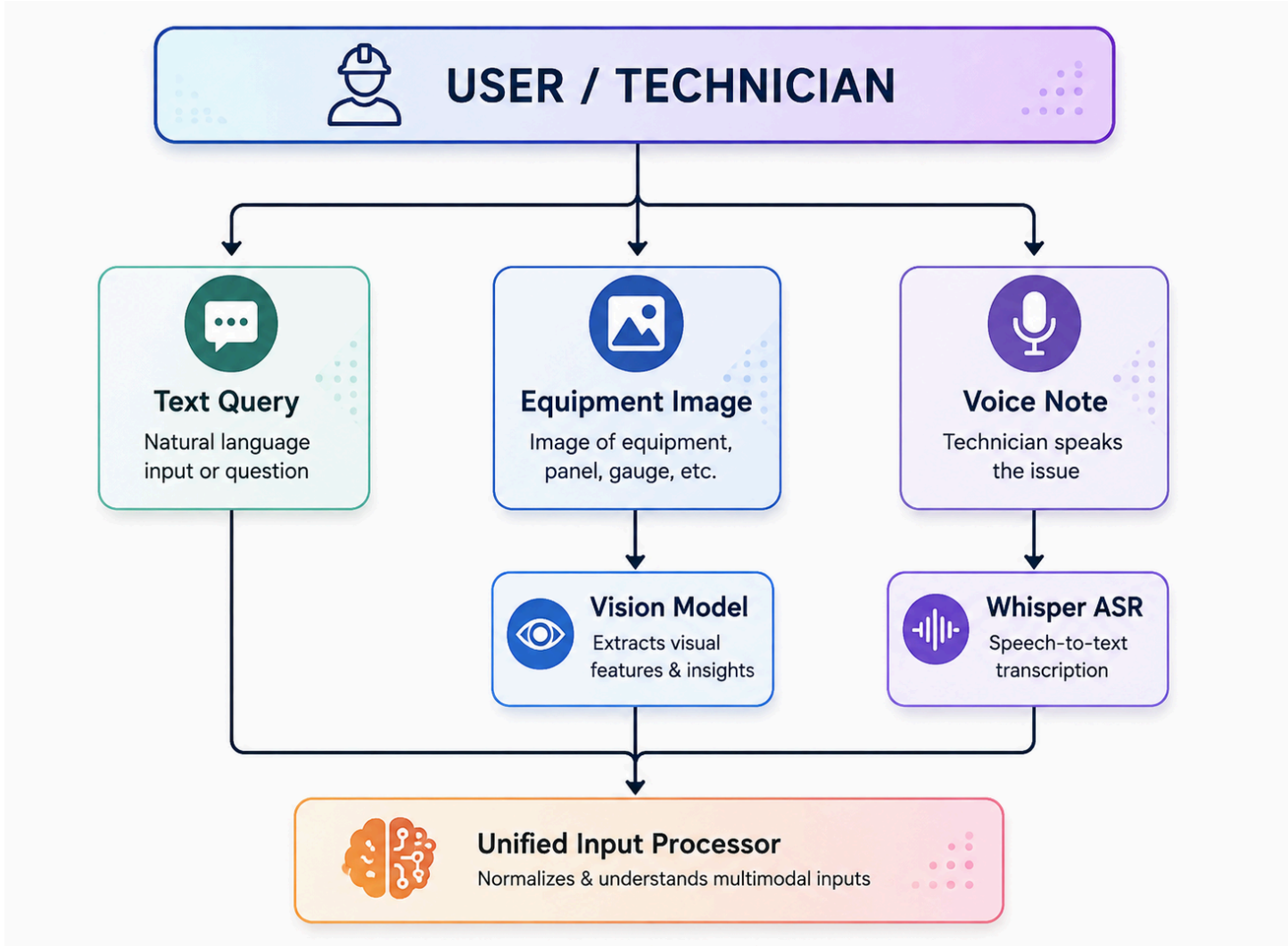
2. SYSTEM OBJECTIVES

The primary objective is to design and implement a secure, multimodal, retrieval-augmented Generative AI assistant that helps industrial technicians diagnose equipment faults using technical documents, equipment images and voice descriptions while maintaining source traceability, security and responsible-AI safeguards.

9. Build a Retrieval-Augmented Generation (RAG) pipeline.
10. Generate semantic embeddings for technical document chunks.
11. Store and retrieve embeddings using FAISS.
12. Retrieve the most relevant technical passages for a query.
13. Implement an AI agent with controlled tools and decision rules.
14. Process text, equipment images and speech input.
15. Fuse modality-specific evidence with retrieved technical knowledge.
16. Generate source-grounded diagnostic guidance.
17. Deploy the application through a Streamlit web interface.
18. Implement authentication, input validation, rate limiting and secure secret management.
19. Maintain audit logs and human-review escalation.
20. Evaluate retrieval, multimodal fusion, agent decisions and security handling.

3. PROPOSED SYSTEM ARCHITECTURE

The architecture integrates the three mandatory subsystems: (i) Knowledge Retrieval and Agent Reasoning, (ii) Multimodal Understanding and Generation, and (iii) Responsible Web Deployment and Governance.



Architecture principle

The system follows an evidence-first design: user inputs are converted into structured evidence, relevant knowledge is retrieved, the agent selects controlled tools, and only then is a diagnostic response generated.

4. PART I — RAG, VECTOR DATABASE AND AI AGENTS

4.1 Question 9 — Design of the RAG Pipeline

4.1.1 RAG Processing Flow



4.1.2 Document Corpus

ID	Document	Type	Purpose
D1	Motor Maintenance Manual	Manual	Motor specifications and maintenance guidance
D2	Motor Overheating SOP	SOP	Safe checks for abnormal motor temperature
D3	Bearing Inspection Procedure	SOP	Bearing wear, noise, vibration and lubrication checks
D4	Pump Failure Incident Report	Incident Log	Historical pump vibration/failure information
D5	Electrical Safety Manual	Safety Manual	Isolation and safe inspection requirements

4.1.3 Chunking Strategy

A fixed-size chunking strategy is selected with approximately 500 words per chunk and 75 words of overlap. Chunks should preserve complete technical statements, headings and safety instructions where possible. Metadata such as document ID, section title, page number and document type should be retained with every chunk.

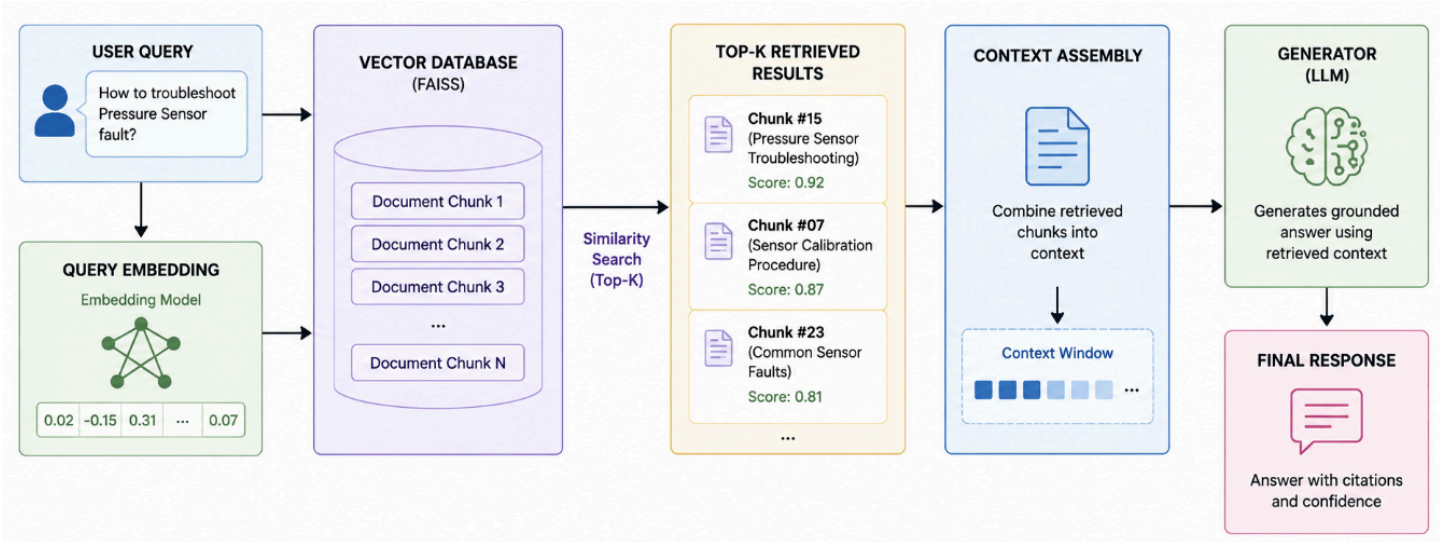
4.1.4 Embedding Model

The proposed embedding model is Sentence Transformers all-MiniLM-L6-v2. It produces 384-dimensional dense embeddings suitable for semantic similarity search. For the final implementation, the model version should be recorded in the project configuration.

4.1.5 Vector Database

FAISS is selected as the vector database/indexing library for the classroom-scale implementation. IndexFlatIP is appropriate when embeddings are normalized because inner-product search then corresponds to cosine similarity.

4.1.6 Retrieval Strategy



4.1.7 Prompt Augmentation

```
SYSTEM ROLE:
You are an industrial equipment diagnostic decision-support
assistant.
RULES:
1. Use the supplied technical evidence.
2. Do not invent specifications or procedures.
3. State uncertainty when evidence is insufficient.
4. Do not recommend unsafe actions.
5. Cite the supporting source.
6. Escalate safety-critical cases to human review.

USER QUERY:
{query}

IMAGE EVIDENCE:
{image_description}

VOICE TRANSCRIPTION:
{voice_text}

RETRIEVED EVIDENCE:
{retrieved_chunks}

TASK:
Generate a concise diagnostic explanation, possible causes,
safe recommended checks, confidence and source citations.
```

Design decision

The LLM should be instructed to distinguish evidence-supported observations from assumptions. An image or generated explanation must not be treated as definitive physical inspection.

4.2 Question 10 — Embedding, Indexing and Retrieval Demonstration

4.2.1 Representative Sample Documents

ID	Document	Representative Chunk
D1	Motor Maintenance Manual	“The motor operating temperature should remain within the manufacturer's specified range. Excessive temperature may indicate overload, poor ventilation, bearing friction or electrical imbalance.”
D2	Motor Overheating SOP	“When motor temperature increases abnormally, stop the equipment using the approved shutdown procedure. Inspect ventilation openings and verify

ID	Document	Representative Chunk
		that the motor is not overloaded.”
D3	Bearing Inspection SOP	“Unusual vibration and noise may indicate bearing wear. Inspect the bearing for abnormal noise, overheating, lubrication problems and mechanical damage.”
D4	Pump Incident Log	“A centrifugal pump experienced increased vibration after extended operation. Inspection identified bearing wear and insufficient lubrication.”
D5	Electrical Safety Manual	“Electrical equipment must be isolated and verified de-energized before personnel perform internal inspection or maintenance.”

4.2.2 Embedding and Indexing Configuration

Parameter	Selected Value
Embedding model	all-MiniLM-L6-v2
Vector dimension	384
Similarity	Inner Product after vector normalization
Index	FAISS IndexFlatIP
Top-k	3
Chunk overlap	Approximately 75 words

Sample Query 1

Query: “Why is the industrial motor overheating?”

Rank	Source	Illustrative Similarity	Relevant Evidence
1	D2 — Motor Overheating SOP	0.82	Overload and ventilation problems can cause overheating.
2	D1 — Motor Maintenance Manual	0.78	Overload, bearing friction and electrical imbalance are possible causes.
3	D3 — Bearing Inspection SOP	0.61	Bearing problems may contribute to overheating and vibration.

4.2.3 Sample Query 2

Query: “The pump has high vibration. What should I check?”

Rank	Source	Illustrative Similarity	Relevant Evidence
1	D4 — Pump Incident Log	0.84	High vibration was associated with bearing wear.
2	D3 — Bearing Inspection SOP	0.80	Inspect bearing condition and lubrication.
3	D1 — Motor Maintenance Manual	0.57	Mechanical friction may contribute to abnormal operation.

Academic integrity note

The similarity scores in this section are illustrative demonstration values. They must be replaced with actual values generated by the implemented FAISS retrieval pipeline before submission as experimental results.

4.3 Question 11 — Simple AI Agent Design

The agent is implemented as a controlled router rather than an unrestricted autonomous system. It determines whether the query requires knowledge retrieval, a maintenance calculation, or clarification.

4.3.1 Agent Tools

Tool	Purpose	Input	Output
Knowledge Retrieval Tool	Search technical knowledge	Technical query	Top-k chunks, sources and similarity
Maintenance Calculator	Calculate maintenance due status	Last service + interval + date	Due / overdue status
Clarification Tool	Request missing information	Missing fields	Clarifying question

4.3.2 Decision Policy

```
IF      technical      documentation      is      required:
CALL      Retrieval      Tool

ELSE    IF      numerical/date      maintenance      calculation      is      required:
CALL      Maintenance      Calculator

ELSE    IF      required      information      is      missing:
ASK      a      Clarifying      Question

ELSE:
GENERATE the response
```

4.3.3 Representative Agent Trace

```
User:
"The motor is overheating. When was its next maintenance due?"

Agent:
[Input received]
[Query classification]
[Technical evidence required] → Retrieval Tool
[Maintenance date required] → Maintenance Calculator
[Evidence + calculation combined]
[Response generated]
[Confidence and safety check]
[Sources and decision trace displayed]
```

Example final response: The available documentation identifies overload, inadequate ventilation, bearing friction and electrical imbalance as possible contributors to motor overheating. The maintenance calculator determines the service status from the supplied service date and maintenance interval. The technician should follow the approved shutdown and inspection procedure. Safety-critical or uncertain cases should be escalated to a qualified engineer.

4.4 Question 12 — Vector Database Indexing Strategies

Strategy	Accuracy	Speed	Memory	Best Use
Flat / Brute Force	Very high	Lower at large scale	Moderate	Small corpus
HNSW	High	Very fast	Higher	Medium to large corpus
IVF	High	Fast	Moderate	Large corpus

Flat search compares the query against every vector and is simple and accurate for small datasets. HNSW uses a graph-based approximate nearest-neighbor structure and provides fast search at the cost of additional memory and indexing complexity. IVF partitions the vector space and searches selected clusters, reducing search cost for large datasets but introducing tuning parameters.

Final indexing decision

FAISS IndexFlatIP is recommended for the initial IndustroSense AI implementation because the expected classroom corpus is small. HNSW or IVF can be considered when the corpus grows substantially.

5. PART II — MULTIMODAL GENERATIVE AI INTEGRATION

5.1 Question 13 — Multimodal Architecture

The system supports three mandatory modalities: text, image and speech. Each modality is processed independently first, then converted into structured evidence for late fusion with the retrieved technical knowledge.

Modality	Input	Model / Component	Output
Text	Technical description	LLM / NLP preprocessing	Structured technical query
Image	Equipment photograph	Vision-language model	Component/condition description
Speech	Voice note	Whisper ASR or equivalent	Text transcription
Knowledge	Manuals/SOPs/logs	Sentence Transformer + FAISS	Retrieved evidence
Generation	Fused evidence	LLM	Grounded diagnostic response

5.1.1 Text Processing

Text → Clean / validate → Technical query → Agent → Retrieval → LLM

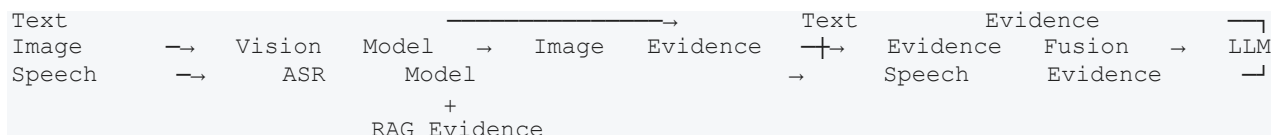
5.1.2 Image Processing

The vision component identifies the visible equipment or component and describes observable conditions such as corrosion, leakage, cracks or unusual external appearance. It should explicitly avoid claiming that an internal failure is confirmed when the image cannot establish it.

5.1.3 Speech Processing

Whisper or an equivalent speech-recognition model converts the voice note into text. The transcription should be treated as an input signal that may contain recognition errors, particularly in noisy industrial environments.

5.1.4 Late-Fusion Strategy



Recommended approach

Late textual fusion is recommended for the student implementation because it is easier to implement, debug and explain, supports missing modalities, and provides clear provenance for each modality.

5.2 Question 14 — Complete End-to-End Example

5.2.1 User Inputs

Input	Example
Text	“The centrifugal pump is producing abnormal vibration.”
Image	Photograph of an industrial centrifugal pump assembly.
Voice	“The vibration has increased after several hours of operation and there is unusual noise.”

5.2.2 Modality-Specific Outputs

Modality	Example Output
Text	Abnormal pump vibration.
Image	The image appears to show a centrifugal pump assembly. No definitive internal failure can be established from the image alone.
Speech	The vibration has increased after several hours of operation and there is unusual noise.
RAG	Bearing wear and insufficient lubrication are documented causes of abnormal vibration.
Agent	Retrieval tool selected for pump and bearing maintenance evidence.

5.2.3 Fused Diagnostic Response

Diagnostic Summary The available evidence suggests that bearing wear or insufficient lubrication should be investigated first. Other mechanical causes cannot be excluded.

Item	Recommendation
Safe checks	Follow approved shutdown procedure; verify load; inspect vibration/noise; inspect bearing condition; verify lubrication.
Confidence	Moderate–High, subject to the supplied equipment information matching the retrieved documentation.
Limitation	The system provides decision-support guidance and does not replace qualified engineering inspection.
Sources	Pump Incident Log; Bearing Inspection SOP; Motor Maintenance Manual.

5.3 Question 15 — Key Engineering Challenges

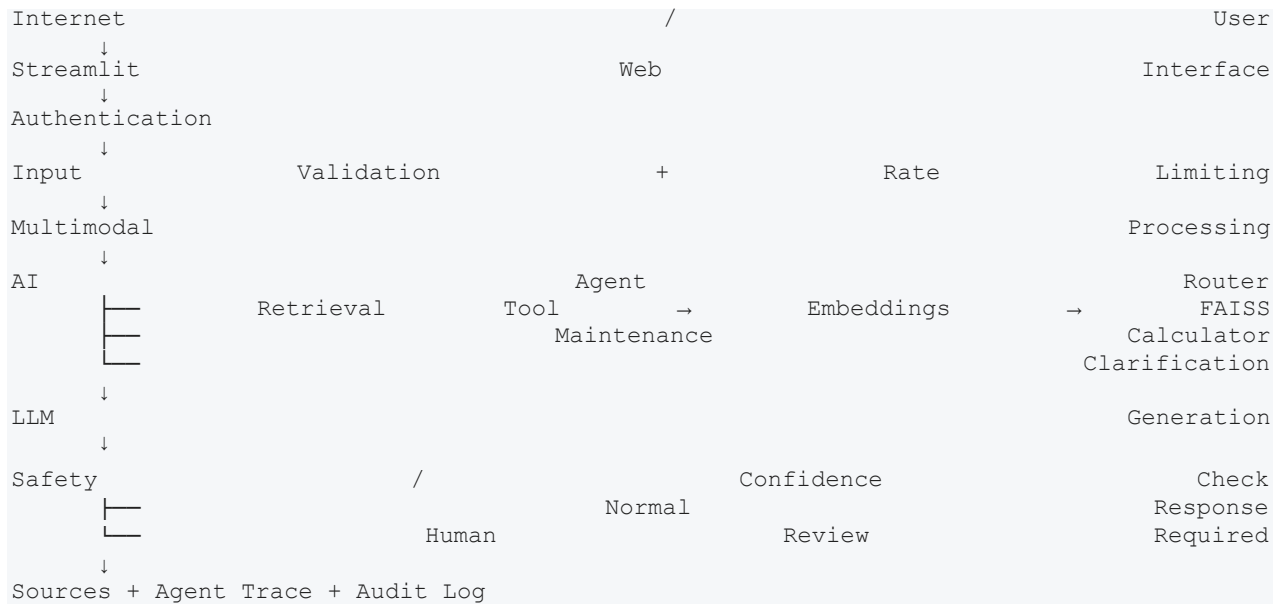
Challenge	Problem	Design Response
Modality alignment	Different modalities describe the same fault differently.	Normalize each output into structured evidence fields.
Missing modality	A user may provide only text.	Allow text-only, image-only, speech-only and combined workflows.
Poor image quality	Blurred or dark images reduce visual reliability.	Image-quality checks and uncertainty messaging.
Speech noise	Factory noise can reduce transcription accuracy.	ASR confidence/quality checks and confirmation where necessary.
Latency	Multiple model calls increase response time.	Use lightweight models, caching and bounded input sizes.
Model interoperability	Different models produce different formats.	Use a common intermediate evidence schema.
Conflicting evidence	Text, image and speech may disagree.	Show the conflict and escalate when it affects safety.
Hallucination	The LLM may infer unsupported faults.	RAG grounding, evidence thresholds and human review.

6.

PART III — RESPONSIBLE DEPLOYMENT, SECURITY AND GOVERNANCE

6.1 Question 16 — Web Deployment Architecture

The application is designed as a lightweight Streamlit web system. A production-scale version could place a FastAPI backend between the interface and inference services, but the student implementation can keep the architecture compact while preserving authentication, validation, logging and governance controls.



6.1.1 Proposed Technology Stack

Layer	Technology
Front end	Streamlit
Application logic	Python
RAG	Custom Python RAG pipeline
Embeddings	Sentence Transformers
Vector index	FAISS
LLM	Accessible LLM/API or open-source equivalent
Vision	Vision-language model/API
Speech	Whisper or equivalent ASR
Logging	Python logging / SQLite

Authentication	OIDC or application-level authentication
----------------	--

Layer	Technology
Version control	Git / GitHub

6.2 Question 17 — Responsible-AI Risks and Mitigation

Risk	Impact	Technical / Process Safeguard
Hallucinated diagnostic advice	Incorrect maintenance decision	RAG grounding, citations, confidence threshold, safety check and human review.
Bias / incomplete knowledge	Unbalanced or incomplete recommendations	Curated corpus, source versioning, multiple-source comparison and periodic review.
Privacy leakage	Exposure of personnel or plant information	Data minimization, temporary storage, access control and deletion of unnecessary media.
Prompt injection	Model manipulation or unauthorized disclosure	Input filtering, strong system instructions, tool allowlisting and source isolation.
Overreliance on AI	Technician treats output as definitive	Visible uncertainty, limitations and human-review escalation.

6.3 Question 18 — Security Considerations

6.3.1 Authentication and Access Control

Authorized users should authenticate before accessing the system. Technician accounts should be limited to diagnostic functions, while administrator access should include document management, configuration and audit review.

6.3.2 Input Validation

Input	Controls
Text	Maximum length, empty-input rejection, prompt-injection screening and safe encoding.
Image	JPG/JPEG/PNG allowlist, MIME validation, size limit, temporary storage and executable-file rejection.
Audio	WAV/MP3/M4A allowlist, MIME validation, size/duration limits and temporary storage.

6.3.3 Rate Limiting

An illustrative classroom policy is a maximum of 10 requests per minute per authenticated user. The exact value should be adjusted according to the available API limits and deployment capacity.

6.3.4 Secure API-Key Management

- Never hard-code API keys in Python source code.
- Store secrets in environment variables or Streamlit secrets.
- Add .env and secret files to .gitignore.
- Rotate compromised keys immediately.
- Do not print secret values in logs.

6.3.5 Audit Logging

Field	Example
Timestamp	2026-09-02 14:30
User ID	technician01
Query ID	Q1024
Input type	text + image
Retrieved sources	D2, D3
Agent action	retrieval
Confidence	0.82
Review flag	No

Privacy principle
Auditability should not become unnecessary data retention. Prefer structured metadata and short-lived media over permanent storage of raw images and audio.

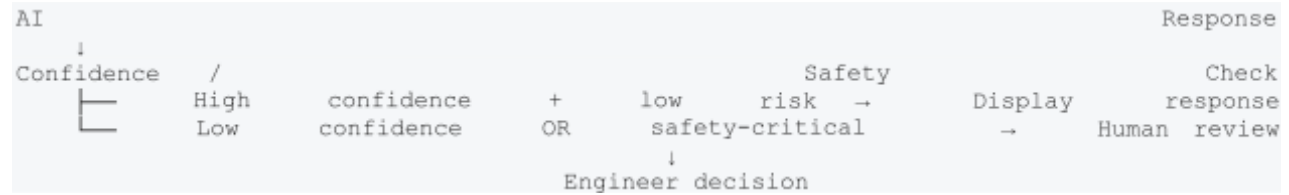
6.4 Question 19 — Governance Considerations

6.4.1 Model Card

Field	IndustroSense AI Description
Purpose	Industrial equipment diagnostic decision support
Capabilities	Technical retrieval, text analysis, image analysis, speech transcription and response generation
Limitations	Cannot physically inspect equipment; image and speech quality affect reliability; generated content can contain errors

Field	IndustroSense AI Description
Human oversight	Required for low-confidence and safety-critical outputs
Data handling	Process only data required for the diagnostic task and minimize retention
Versioning	Track model, prompt, corpus, agent policy and application versions

6.4.2 Human-in-the-Loop Escalation



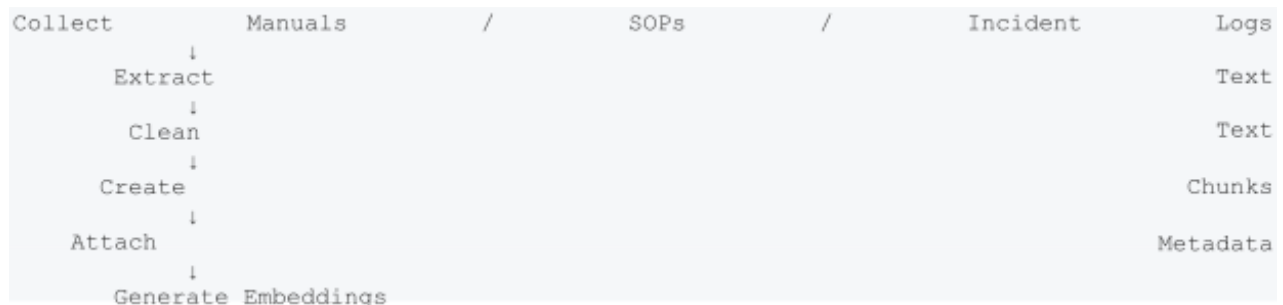
6.4.3 Version Control

Application:	IndustroSense	AI	v1.0
Embedding:			all-MiniLM-L6-v2
Prompt:			diagnostic_prompt_v1.2
Corpus:			industrial_docs_v1.0
Agent			router_v1.1
Security policy:	security_v1.0	policy:	

7. CONSOLIDATED SOLUTION / DESIGN / METHODOLOGY

The three parts are integrated into one evidence-driven workflow. The following methodology can be used as the implementation plan.

7.1 Phase 1 — Knowledge Preparation



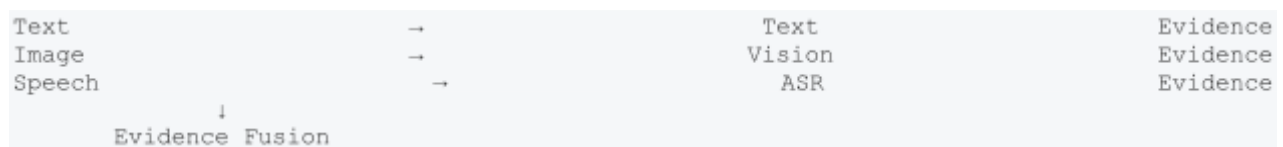
7.2 Phase 2 — RAG Construction

Chunks → Embeddings → FAISS → Similarity Search → Top-3 Evidence

7.3 Phase 3 — Agent Construction

User Query → Router → Retrieval / Calculator / Clarification → Evidence

7.4 Phase 4 — Multimodal Processing



7.5 Phase 5 — Generation and Safety



8. MODERN TOOLS AND IMPLEMENTATION PLAN

Tool / Technology	Role in the Project	Evidence to Include
Python	Core application and RAG implementation	Source code + console output
Streamlit	Interactive web interface	Running application screenshots
FAISS	Vector indexing and retrieval	Indexing/retrieval output
Sentence Transformers	Embedding generation	Embedding configuration
LLM	Grounded response generation	Prompt/output screenshot
Vision-language model	Image analysis	Image input/output screenshot
Whisper / ASR	Speech-to-text	Voice transcription screenshot
Git / GitHub	Version control and submission	Repository link + commit history

8.1 Suggested Project Structure

```

IndustroSense-AI/
├── app.py
├── requirements.txt
├── README.md
├── .gitignore
├── data/
├── manuals/
├── sops
├── /
├── incident_logs/
├── models
├── /
├── configuration/
├── rag/
├── document_loader.py
├── chunker.py
├── embeddings.py
├── vector_store.py
├── retriever.py
├── agent/
├── router.py
├── retrieval_tool.py
├── maintenance_tool.py
├── multimodal/
├── vision.py
├── speech.py
├── fusion.py
├── security/
├── validation.py
├── auth.py
├── rate_limit.py

```


9. RESULTS AND VALIDATION

Important

The assignment requires actual representative outputs. Tables containing numerical similarity, confidence or accuracy values must be populated with the values produced by the student's implementation. The examples below are structured templates.

9.1 Retrieval Quality

Query	Correct Source in Top-3?	Top-1 Relevance	Top-3 Relevance	Result
Motor overheating	To be measured	Actual result	Actual result	Replace with experiment
Pump vibration	To be measured	Actual result	Actual result	Replace with experiment
Bearing noise	To be measured	Actual result	Actual result	Replace with experiment

9.2 Retrieval Metric

A suitable retrieval metric is Precision@K:

$$\text{Precision@K} = \text{Number of relevant retrieved documents} / K$$

For example, if all three retrieved documents are judged relevant, $\text{Precision@3} = 3/3 = 1.00$ or 100%. This example demonstrates the calculation only; the final report should use actual relevance judgments.

9.3 Multimodal Test Case 1

Input	Example
Text	Motor is overheating.
Image	Motor housing photograph.
Speech	The motor becomes very hot after continuous operation.
Expected result	Grounded analysis of overload, ventilation and related evidence with safety guidance.

9.4 Multimodal Test Case 2

Input	Example
Text	Pump vibration is increasing.

Input	Example
Image	Pump assembly photograph.
Speech	The pump makes abnormal noise during operation.
Expected result	Grounded investigation of bearing/lubrication and other mechanical causes.

9.5 Agent Validation

Test						query:
"When	should	this	motor	be		served?"
Expected						trace:
Query						received
→	Maintenance-related				query	detected
→	Maintenance				Calculator	selected
→	Maintenance				interval	calculated
→				Result		
→				Explanation		
→ Sources / trace displayed						generated

9.6 Security Validation

Test	Input	Expected Behaviour
Empty query	Blank	Reject and request valid input
Oversized image	>5 MB	Reject
Invalid file	.exe	Reject
Prompt injection	Ignore previous instructions	Detect/neutralize and continue safely
Missing modality	Text only	Process without crashing
Low confidence	Weak evidence	Flag for human review
API failure	Unavailable service	Graceful fallback / user-facing error

10. ANALYSIS AND ENGINEERING DECISIONS

10.1 Vector Database Comparison

Factor	Flat	HNSW	IVF
Implementation complexity	Low	Medium	Medium
Small corpus	Excellent	Good	Good
Search speed	Moderate	High	High
Memory efficiency	Good	Lower	Good
Accuracy	Very high	High	High
Project recommendation	Yes	Future option	Future option

FAISS IndexFlatIP is selected because the initial corpus is small and the project prioritizes simplicity, reproducibility and reliable retrieval. For a production-scale corpus, HNSW or IVF can reduce retrieval latency.

10.2 Multimodal Fusion Comparison

Factor	Early Fusion	Late Fusion
Implementation	Complex	Simple
Debugging	Difficult	Easy
Missing modality handling	More difficult	Straightforward
Provenance	Less transparent	Clear per modality
Compute needs	Higher	Moderate
Recommendation	Future research	Recommended for project

10.3 Responsible-AI Trade-off

Automatic responses provide low latency and low operational cost, but they can expose technicians to unreviewed hallucinated or unsafe advice. Human review increases latency and requires engineering resources but provides a stronger safety barrier. Therefore, a risk-based escalation mechanism is recommended: low-risk, high-confidence cases can be displayed directly, while low-confidence or safety-critical cases are routed for human review.

10.4 Overall Engineering Decision

- FAISS IndexFlatIP for the initial small corpus.

- Late textual multimodal fusion for implementation simplicity and traceability.
- Controlled rule-based agent routing for predictable tool selection.
- Top-3 retrieval with a calibrated relevance threshold.
- Evidence-grounded prompts with explicit uncertainty handling.
- Risk-based human review for safety-critical or low-confidence outputs.
- Streamlit for a lightweight classroom web deployment.

11. BROADER CONSIDERATIONS

11.1 Reliability and Safety

IndustroSense AI should never present generated content as an unquestionable engineering diagnosis. The response should pass through evidence, confidence and safety checks. If evidence is weak, conflicting or safety-critical, the system should request clarification or escalate to a human engineer.

11.2 Sustainability and Economics

- Retrieve only relevant chunks instead of sending entire documents to the LLM.
- Cache document embeddings so that documents are not repeatedly embedded.
- Use bounded top-k retrieval.
- Resize images before vision processing.
- Limit audio duration and file size.
- Reuse loaded models where possible.
- Use an appropriately sized model for the task.

11.3 Accessibility and Society

Multimodal interaction improves accessibility by allowing technicians to report faults using text, photographs or speech. Speech can reduce typing requirements, while images can communicate physical conditions that are difficult to describe verbally.

11.4 Ethics and Professional Responsibility

- Collect only information required for diagnosis.
- Protect plant images and voice recordings.
- Minimize retention of raw media.
- Do not expose confidential maintenance information to unauthorized users.
- Communicate uncertainty clearly.
- Maintain human responsibility for safety-critical decisions.
- Keep model, prompt and knowledge-corpus versions traceable.

12. CONCLUSION

IndustroSense AI provides an integrated solution for multimodal industrial equipment diagnostics by combining Retrieval-Augmented Generation, vector databases, AI-agent reasoning, vision-language processing, speech recognition and responsible web deployment.

The proposed RAG pipeline converts technical manuals, SOPs and incident records into searchable vector representations using a lightweight sentence-embedding model and FAISS. The retrieval layer supplies relevant evidence to the language model and exposes source provenance to improve transparency.

The AI-agent layer introduces controlled decision-making through retrieval and maintenance-calculation tools. The multimodal subsystem processes text, images and speech independently before combining their outputs with retrieved technical evidence. This late-fusion approach provides a practical balance between implementation complexity, latency, interpretability and robustness.

The responsible-deployment layer adds authentication, input validation, secure secret management, audit logging, confidence indicators, prompt-injection safeguards and human-in-the-loop escalation. These controls are essential because industrial diagnostic systems can influence safety-related decisions.

Overall, the design addresses the major requirements of the assignment: RAG and vector-based retrieval, agent reasoning, multimodal input, Streamlit deployment, provenance, security, governance and evaluation. Actual implementation screenshots and measured results should be inserted before final submission.

13. STUDENT REFLECTION

13.1 What I Learned

Developing IndustroSense AI provides practical understanding of how RAG, vector databases, multimodal models, AI agents and responsible deployment can be integrated into one complete application. I learned that RAG reduces dependence on unsupported model memory by supplying relevant external evidence. I also understood the importance of embeddings and similarity search for retrieving technically related information.

The multimodal design demonstrated that text, image and speech processing require careful coordination and validation. Most importantly, the project showed that industrial Generative AI requires more than response quality: it must communicate uncertainty, provide evidence, protect sensitive information and include human oversight for critical decisions.

13.2 Future Improvements

21. A larger and more diverse industrial document corpus.
22. A production-grade vector database.
23. Hybrid keyword + semantic retrieval and reranking.
24. Domain-specific embedding models.
25. More robust industrial vision models.
26. Improved multilingual speech support.
27. Formal red-team testing and security assessment.
28. Integration with real-time equipment sensor data.
29. Automated faithfulness and hallucination evaluation.
30. Enterprise-grade monitoring and role-based access control.

14. REFERENCES

- [1] E. Tabassi, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” National Institute of Standards and Technology, NIST AI 100-1, 2023.
- [2] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” OWASP GenAI Security Project, 2025.
- [3] OWASP Foundation, “LLM01:2025 Prompt Injection,” OWASP GenAI Security Project, 2025.
- [4] OWASP Foundation, “LLM02:2025 Sensitive Information Disclosure,” OWASP GenAI Security Project, 2025.
- [5] Sentence Transformers, “all-MiniLM-L6-v2,” Hugging Face Model Card, accessed 2026.
- [6] Meta AI Research, “FAISS Documentation,” FAISS documentation, accessed 2026.
- [7] OpenAI, “Whisper: Robust Speech Recognition via Large-Scale Weak Supervision,” GitHub repository and model documentation, accessed 2026.
- [8] Streamlit, “User Authentication,” Streamlit Documentation, accessed 2026.
- [9] Streamlit, “Secrets Management,” Streamlit Documentation, accessed 2026.
- [10] CSA6502 — Generative AI and Large-Scale Models, Common Course Assignment: Multimodal Responsible Generative AI Application with Web Deployment, Academic Year 2026–2027.